

Änderungsprotokoll — Adressverwaltung v2.3.0

Datum: 4. Oktober 2026
Ausgangsstand: v2.2.0 (Commit 1b21258)
Endstand: Commit f09a2fc auf main, 8 Commits, 69 geänderte Dateien
Versionsnummer: 2.3.0 (vorher 2.2.0)

1. Übersicht

Version 2.3.0 fasst die Änderungen vom 4. Oktober 2026 zusammen: fünf Änderungspakete und zwei neue Dokumente.

#	Paket	Inhalt	Commit
1	Backend-Tests	Neues Testprojekt mit Integrationstests für die OData-API	da8d488
2	NuGet-Sicherheitsupdates	Pakete mit bekannten Lücken aktualisiert	45c0592
3	.NET 10	Backend und Testprojekt von .NET 8 auf .NET 10 migriert	243d6fc
4	Sicherheit	18 Befunde geprüft, 17 behoben; Next.js 15 und React 19; Bericht als PDF	e5cf8ee
5	Clean Code	Nachprüfung mit 11 Befunden (B-09 bis B-13, F-07 bis F-12), alle umgesetzt	dac15cb
6	Dokumentation	PDF der Clean-Code-Analyse neu erzeugt	6d1975d
7	Dokumentation	Schulungsunterlage zum Sequenzdiagramm	080cb4a, f09a2fc
8	Version	Versionsnummer auf 2.3.0 gesetzt (README, package.json, Diagramme, Dokumente)	nach f09a2fc

Das Datenbankschema ist unverändert. Bestehende Daten bleiben erhalten.

Für den Betrieb ändert sich einiges; Abschnitt 8 beschreibt, was nach dem Aktualisieren zu tun ist.

2. Backend-Tests

Neu: backend/AdressverwaltungApi.Tests/ und backend/Adressverwaltung.sln

xUnit-Integrationstests für die OData-API. Die API läuft im Speicher (webApplicationFactory<Program>) gegen eine echte PostgreSQL-Datenbank, die pro Testlauf als Wegwerf-Container startet (Testcontainers). Der E-Mail-Versand ist durch FakeEmailService ersetzt.

Datei	Inhalt
AdressenODataTests.cs	CRUD, Validierung, Audit-Felder, OData-Abfragen, Benachrichtigung
CitiesODataTests.cs	CRUD und PLZ-Suche
AuthorizationTests.cs	Endpunkte nur mit gültigem Token erreichbar
AuthSecurityTests.cs	Registrierung nur mit Token, gleiche Antworten, X-User wird ignoriert

Datei	Inhalt
MetadataTests.cs	\$metadata und camelCase
Infrastructure/	ApiFactory, ODataTestBase, FakeEmailService

Zwei Änderungen an Program.cs waren dafür nötig: `public partial class Program { }` macht die Klasse für das Testprojekt sichtbar, und `NameClaimType = "name"` sorgt dafür, dass `CreatedBy/ChangedBy` aus dem Token gefüllt werden. Vorher stand dort immer `system`.

Aufruf: `cd backend && dotnet test` (Docker muss laufen). Stand: 51 Tests, alle bestanden.

3. Abhängigkeiten und Plattform

Backend

Komponente	vorher	nachher
Zielframework (API und Tests)	net8.0	net10.0
Docker-Images	sdk:8.0, aspnet:8.0	sdk:10.0, aspnet:10.0
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.0	10.0.12
Microsoft.EntityFrameworkCore.Design	8.0.7	10.0.12
Npgsql.EntityFrameworkCore.PostgreSQL	8.0.4	10.0.3
Microsoft.AspNetCore.OData	8.2.5	9.5.0
Swashbuckle.AspNetCore	6.6.2	10.2.3

Der Zwischenschritt in Paket 2 hob die Pakete zunächst innerhalb von .NET 8 an und beseitigte vier Warnungen zu transitiven Paketen (`Microsoft.Extensions.Caching.Memory`, `System.Text.Json`, `System.IdentityModel.Tokens.Jwt`, `Microsoft.IdentityModel.JsonWebTokens`).

Frontend

Komponente	vorher	nachher
next, eslint-config-next	14.2.5	15.5.27
react, react-dom	18	19
postcss	8.4.x (in next gebündelt)	≥ 8.5.23 (per overrides)
@types/next-auth	3.15	entfernt (veraltet)
package.json Version	2.0.0	2.3.0
Lockfile	keines	package-lock.json, Dockerfile nutzt <code>npm ci</code>

Next.js 14 erhält für mehrere kritische Lücken keine Korrekturen mehr, deshalb der Sprung auf Version 15. Am Anwendungscode war dafür keine Anpassung nötig.

4. Sicherheit

Der vollständige Bericht liegt in `documentation/Sicherheitsbericht.pdf`. Hier die Änderungen nach Bereich.

Backend

Datei	Änderung
Controllers/AuthController.cs	/auth/register nur noch mit Token; gleiche Antwort für neue und bestehende E-Mail; fester Vergleichs-Hash gegen Zeitmessung beim Login
Dtos/AuthDtos.cs	Höchstlängen für E-Mail (256) und Passwort (128)
Data/AdresseDbContext.cs	Header X-User wird für Audit-Felder nicht mehr gelesen
DbSeeder.cs	Kein festes Standardpasswort mehr; ohne Seed:AdminPassword Zufallspasswort im Log
Program.cs	Start bricht ab, wenn Jwt:Key fehlt, zu kurz oder ein Platzhalter ist
appsettings.json	JWT-Schlüssel und Datenbankpasswort entfernt
Dockerfile	Container läuft als Benutzer app statt root

Frontend

Datei	Änderung
app/login/LoginForm.tsx	callbackUrl nur noch innerhalb der eigenen App (kein Open Redirect)
lib/api.ts	Hochkomma in der PLZ-Suche wird maskiert
Dockerfile	npm ci mit Lockfile; Container läuft als Benutzer node

Infrastruktur und Skripte

Datei	Änderung
docker-compose.yml	Geheimnisse entfernt, stattdessen env_file; Backend und PostgreSQL nur an 127.0.0.1
env/*.env.example	Neue Vorlagen für db.env, backend.env, frontend.env (die echten Dateien sind git-ignoriert)
nginx/nginx.conf	Ratenbegrenzung für Anmeldungen (10 pro Minute und IP); zusätzliche Sicherheits-Header; Version nicht mehr sichtbar
scripts/create_user.sh, scripts/reset_password.sh	Werte als psql-Variablen statt im SQL-Text; Passwort nicht mehr ausgegeben; Mindestlänge 8
migration/import_cities.py	Passwort aus dem Kommentar entfernt (Datei in Paket 5 gelöscht)

5. Clean Code

Befunde und Begründungen stehen in der Nachprüfung am Ende von `documentation/CleanCode_Analyse.md`.

Backend

#	Datei	Änderung
B-09	neu Options/JwtOptions.cs, Options/SmtpOptions.cs; Program.cs, Services/EmailService.cs	Typisierte Konfiguration statt Zugriff über Zeichenketten
B-10	neu Services/ITokenService.cs, Services/JwtTokenService.cs; AuthController.cs	JWT-Erzeugung aus dem Controller ausgelagert; E-Mail-Suche in FindUserByEmailAsync
B-11	odataCrudController.cs, AdressenController.cs	Erweiterungspunkt OnCreatedAsync statt kopierter Post-Methode; ungenutztes Feld entfernt
B-12	Data/AdresseDbContext.cs	Doppelte Fluent-Konfiguration entfernt; erzeugtes Schema identisch
B-13	Dtos/SettingsDtos.cs, SettingsController.cs	DTO SettingsResponse statt anonymer Objekte

Frontend

#	Datei	Änderung
F-07	lib/api.ts	Hilfsfunktionen apiRequest, jsonRequest, leseListe; Konstanten für die PLZ-Suche
F-08	types/auditable.ts, types/adresse.ts, types/city.ts	Generische Typen CreateOf<T> und UpdateOf<T>
F-09	neu components/formStyles.ts, Fehlermeldung.tsx, Ladeanzeige.tsx; Formulare und Seiten	Gemeinsame Formular-Klassen, Fehlerbox und Ladeanzeige
F-10	app/cities/DeleteButton.tsx, app/adressen/[id]/bearbeiten/page.tsx, ConfirmDialog.tsx	Dialog und Fehler-State statt confirm()/alert()
F-11	components/CityForm.tsx, AdresseForm.tsx	Controlled Inputs; laden in speichert umbenannt
F-12	components/NavBar.tsx	Link «Adressen» zeigte auf eine nicht vorhandene Seite; führt jetzt auf die Liste

Gelöscht: migration/import_cities.py, eine identische Kopie von scripts/import_cities.py.

6. Dokumentation

Datei	Stand
documentation/Sicherheitsbericht.pdf (Quelle .html)	neu
documentation/Schulungsunterlage_Sequenzdiagramm.pdf (Quelle .html, zwei PNG)	neu
documentation/Aenderungsprotokoll_v2.3.0.pdf (Quelle .md)	neu, dieses Dokument
documentation/CleanCode_Analyse.md und .pdf	um die Nachprüfung ergänzt, PDF neu erzeugt
documentation/Systemarchitektur.puml und .svg	ASP.NET Core 10, Next.js 15

Datei	Stand
README.md	Versionen, Geheimnisse in env/, Anmeldung, lokale Entwicklung, Tests, Projektstruktur
CLAUDE.md	an alle Änderungen angepasst

7. Geänderte Dateien nach Bereich

Bereich	neu	geändert	gelöscht
Backend API (backend/AdressverwaltungApi)	4	13	0
Backend-Tests und Solution	10	0	0
Frontend (frontend)	4	17	0
Infrastruktur (docker-compose.yml, nginx, env, .gitignore)	3	3	0
Skripte und Migration	0	2	1
Dokumentation, README.md, CLAUDE.md	6	6	0

Die Zahlen beziehen sich auf den Stand vor diesem Protokoll und vor dem Setzen der Versionsnummer (Commit f09a2fc).

8. Nach dem Aktualisieren zu tun

1. **Geheimnisse anlegen.** Die drei Dateien env/db.env, env/backend.env und env/frontend.env aus den Vorlagen *.env.example erstellen und die Platzhalter ersetzen. Ohne sie startet der Stack nicht.
2. **Datenbankpasswort bei bestehender Datenbank.** POSTGRES_PASSWORD wirkt nur beim ersten Start mit leerem Volume. Bei einem bestehenden Volume das Passwort in der Datenbank mit ALTER USER postgres PASSWORD '...' auf den Wert aus env/db.env setzen.
3. **Images neu bauen.** docker compose up --build -d.
4. **Neu anmelden.** Mit neuem JWT-Schlüssel und neuem NEXTAUTH_SECRET sind bestehende Sitzungen ungültig.
5. **Admin-Passwort.** In einer bestehenden Datenbank gilt das frühere Standardpasswort weiter. Mit ./scripts/reset_password.sh ersetzen.
6. **Lokale Entwicklung.** .NET SDK 10 installieren. dotnet run braucht Jwt__key und ConnectionStrings__DefaultConnection aus der Umgebung. Im Frontend npm ci statt npm install.
7. **Benutzer anlegen.** Neue Benutzer entstehen über ./scripts/create_user.sh oder durch einen angemeldeten Benutzer über /auth/register.

Auf dem Entwicklungsrechner sind die Punkte 1 bis 4 bereits erledigt.

9. Prüfung

Prüfung	Ergebnis
dotnet test	51 von 51 bestanden

Prüfung	Ergebnis
<code>dotnet list package --vulnerable --include-transitive</code>	keine anfälligen Pakete
<code>npm audit --omit=dev</code>	0 Lücken
Frontend-Build mit Typprüfung und Lint im Docker-Image	erfolgreich
Erzeugtes Datenbankschema vor und nach B-12	identisch
Docker-Stack: Anmeldung, Seiten, API-Aufrufe (Anlegen, Ändern, Suchen, Löschen) per <code>curl</code>	erfolgreich
Registrierung ohne Token, Seitenschutz, Ratenbegrenzung, Header, Ports, Container-Benutzer	wie beabsichtigt

Nicht im Browser geprüft: die Bedienung der Oberfläche nach dem Upgrade auf React 19 und dem Umbau des Frontends (Formulare, Löschdialog, Weiterleitung nach dem Login).

10. Offene Punkte

- Früheres Standardpasswort des Admin-Benutzers in der bestehenden Datenbank zurücksetzen.
- Geheimnisse in der Git-History: TLS-Zertifikat neu erzeugen, Postman-Vault-Key ersetzen, das Passwort aus dem früheren Skript-Kommentar überall ändern. Ob die History umgeschrieben wird, ist offen.
- OData-Listen haben keine Seitengröße; das verlangt Blättern im Frontend.
- Keine Rollen, keine Kontosperrung, kein Widerruf von Tokens.
- `npm audit` meldet 7 Einträge in Entwicklungswerkzeugen (über `braces`); behebbar mit Tailwind CSS 4.

Einzelheiten zu den Sicherheitspunkten stehen in `documentation/Sicherheitsbericht.pdf`, Abschnitt 4.